

COMP 1005/1405B

Winter 2022 - Tutorial #1

Objectives

- Practice installing Python interpreter
- Practice installing Visual Studio Code (VS Code)
- Practice writing/running Python code in VS Code
- Practice creating a zip file and submitting on Brightspace
- Practice basic problem solving and computational thinking

Expectations

- This is an individual assignment. You must complete and submit all required exercises individually.
- To receive full grades for this tutorial, you must complete problems 1-4 by 12:59 pm on Jan. 26, 2021.
- Problems 5 and 6 are for practice. If you do not have time to complete the problems, you should attempt them after the tutorial for your own benefit.

Submit

For this tutorial, you will submit a single zip file called **YourStudentNumber_T1.zip** to Brightspace, where **YourStudentNumber** should be replaced by *your student number*. The zip file should contain your solutions to all the required problems.

Grading Scheme

Tutorials account for 10% of your course grade. Each tutorial will be graded out of 1. You will receive 0.5/1 if you complete around 50% of the required problems. You will receive 1/1 for doing the required problems or making significant progress on the problems.

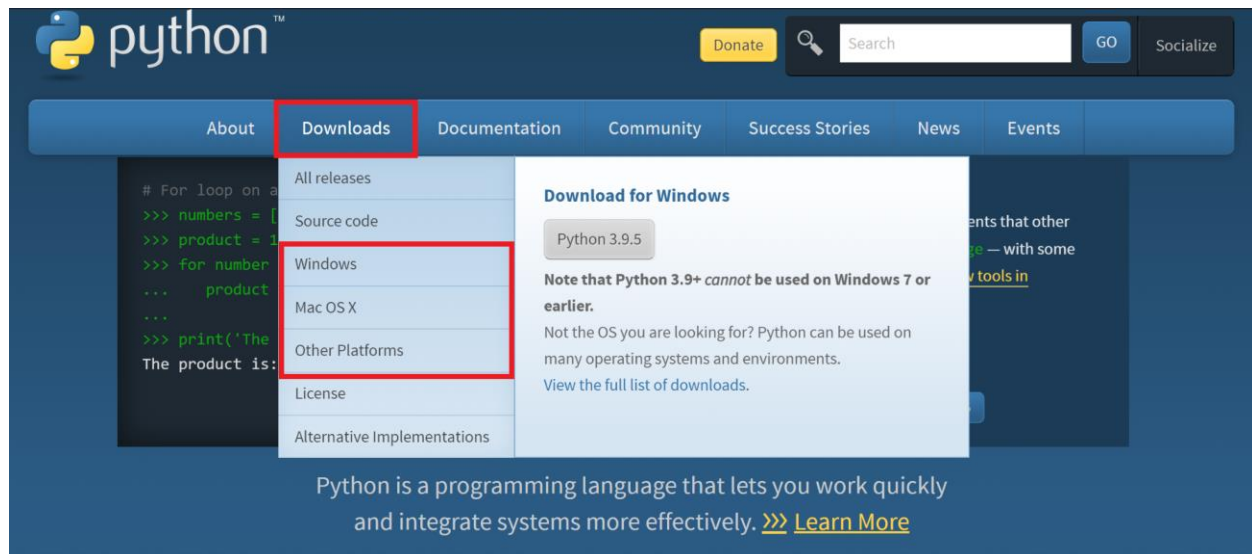
Ensure that your tutorial grade is entered on Brightspace at the end of each week. If you don't see your previous grades, contact the [Tutorial TAs](#) immediately. You have no more than 1 week to report a missing tutorial grade.

Problem 1: Installing Python and getting started with Visual Studio Code

Install Python Interpreter

Download and install the latest version of Python (any version 3.5 or later) from the official website <https://www.python.org/>. Note that if you install Python 2.7, your code may produce incorrect output for this course. The screenshots in this tutorial specification uses Python 3.9.5.

Go to the Downloads drop down menu and select your operating system (Windows, Mac OS, or other)



Select the version of Python 3 you would like to install (recommend the latest version which will be listed right at the top under “Python Releases for <your OS>”)

Python Releases for Windows

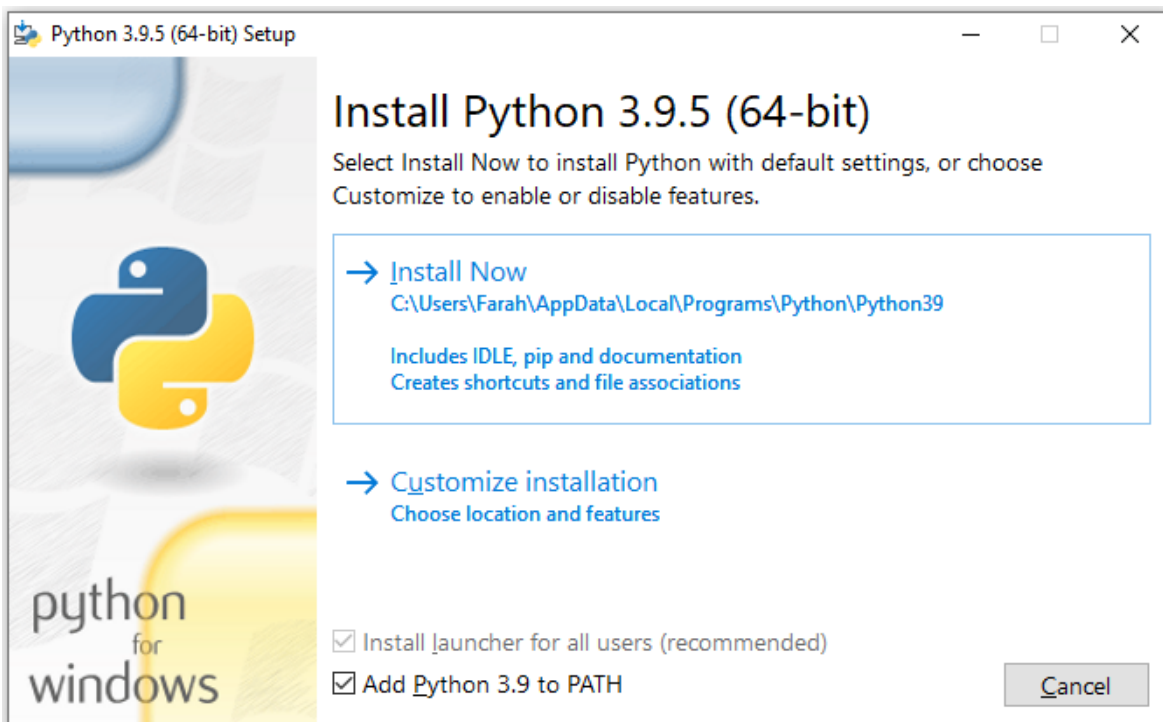
- [Latest Python 3 Release - Python 3.9.5](#)
- [Latest Python 2 Release - Python 2.7.18](#)

Scroll down to “Files” and select the **installer** file for **your OS** with the correct **processor** (**32 bit** or **64 bit**). Below is an example of the file you would select if you have a **Windows 64 bit computer**:

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		364158b3113cf8ac8db7868ce40ebc7b	25627989	SIG
XZ compressed source tarball	Source release		71f7ada6bec9c9dbf4538adc326120cfd	19058600	SIG
macOS 64-bit Intel installer	Mac OS X	for macOS 10.9 and later	870e851eef2c6712239e0b97ea5bf407	29933848	SIG
macOS 64-bit universal2 installer	Mac OS X	for macOS 10.9 and later, including macOS 11 Big Sur on Apple Silicon	59aedbc04df8ee0547d3042270e9aa57	37732597	SIG
Windows embeddable package (32-bit)	Windows		cacf28418ae39704743fa790d404e6bb	7594314	SIG
Windows embeddable package (64-bit)	Windows		0b3a4a9ae9d319885eade3ac5aca7d17	8427568	SIG
Windows help file	Windows		b311674bd26a602011d8baea2381df9e	8867595	SIG
Windows installer (32-bit)	Windows		b29b19a94bbe498808e5e12c51625dd8	27281416	SIG
Windows installer (64-bit)	Windows	Recommended	53a354a15baed952ea9519a7f4d87c3f	28377264	SIG

Open the file and proceed through the installation steps. **Make sure you check the box that says “Add Python 3.x to PATH”** (see the sample screenshot below). The rest of the installation step should be straight forward.

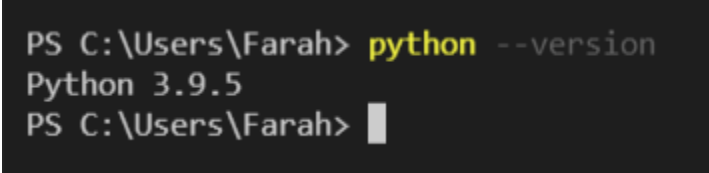


Recommended: To ensure that Python has installed properly, open either a command line, or Powershell, or terminal, and enter:

```
python --version
```

If it installed correctly, you should see the python version displayed on the terminal.

Example:



```
PS C:\Users\Farah> python --version
Python 3.9.5
PS C:\Users\Farah> █
```

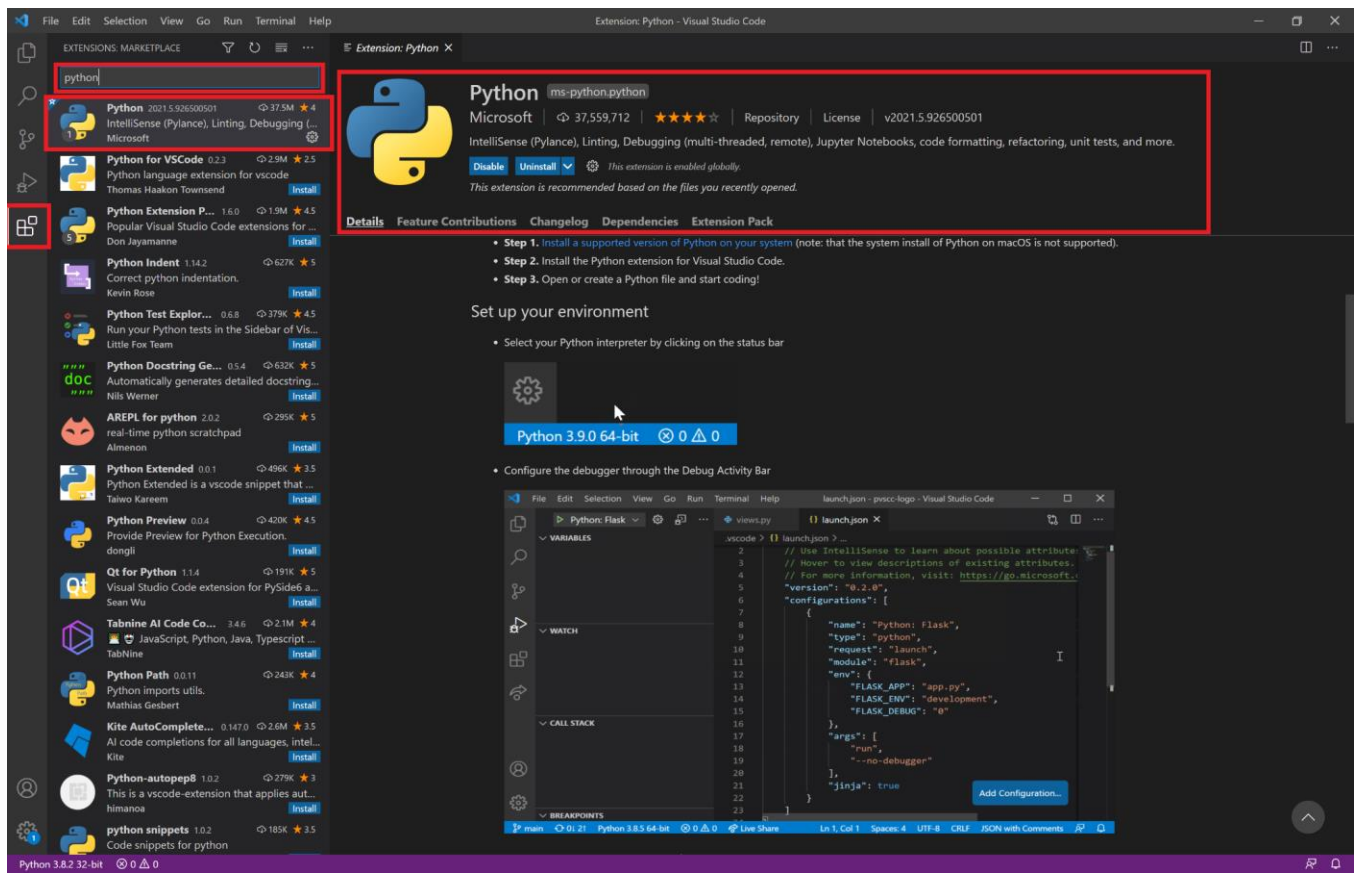
Install and Set Up VS Code

Next download and install VS Code from this site
<https://code.visualstudio.com/download>.

- Simply select the download file for your OS
- Open the installer (the file)
- Follow through the installation process

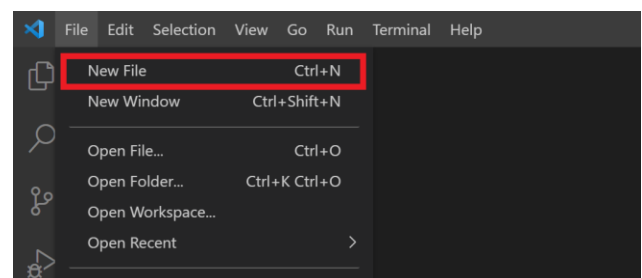
Once VS Code has been installed, open VS Code and install the Python extension (recommended):

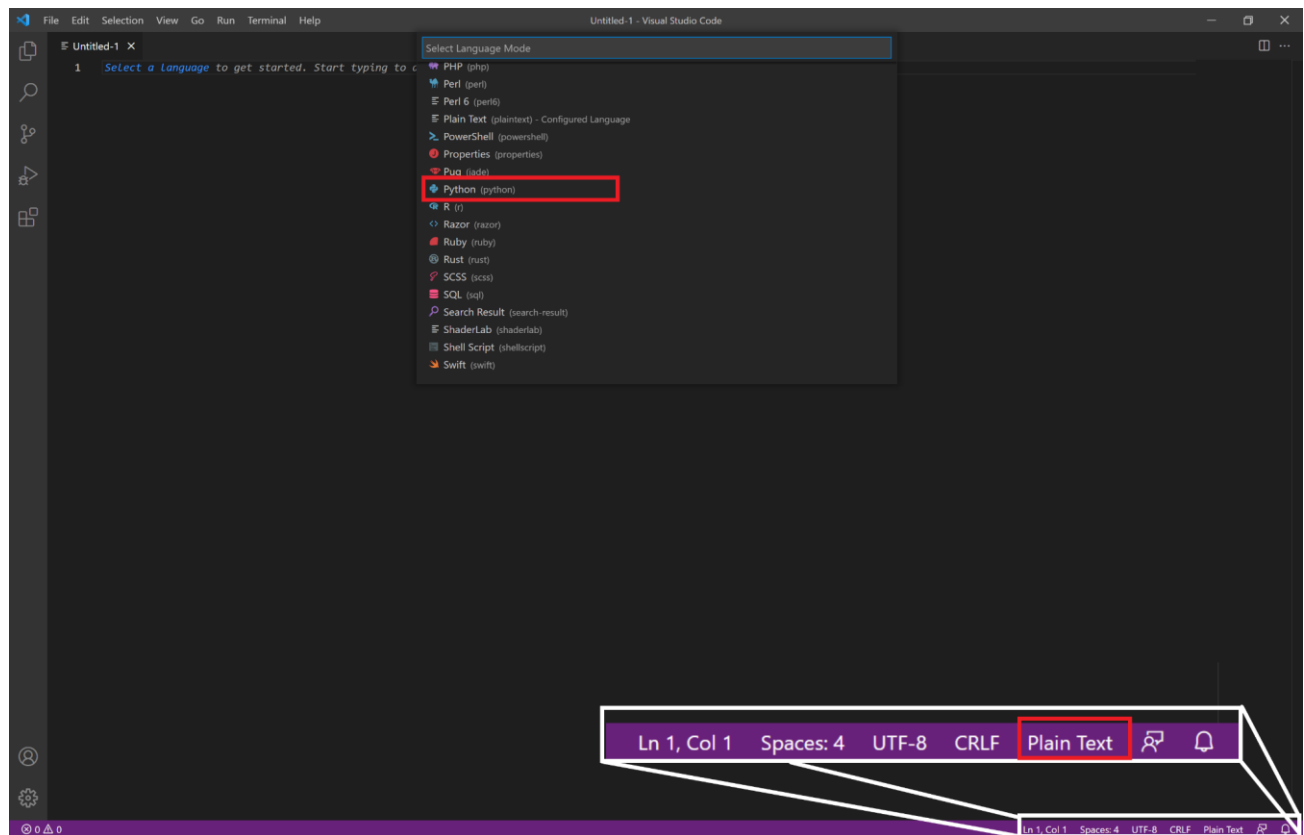
- In VS Code, select the extensions icon on the left (Ctrl + Shift + x)
- Search for “python” and install the extension (below shows the Microsoft extension)



Lastly, you need to make sure your VS Code is using your Python 3.x interpreter (notice at the bottom left corner of the image above it reads “Python 3.9.5 64-bit” - this is the interpreter VS Code is currently using.

To make sure your VS Code can use your Python interpreter, create a new file and in the bottom right corner where it will likely read “Plain Text”, select this button and change the interpreter to Python.





The correct Python 3.x interpreter should now be displayed in the bottom left corner, and you are ready to start programming in Python.



Problem 2 (Your First Python Program)

The following process can be used to solve many programming problems. The steps below outline a process that you can apply throughout this course when working to solve programming problems:

1. Analyze and understand the problem
 - a. What is the problem asking?
 - b. What information is required to solve the problem?
 2. Create a plan to solve the problem
 3. Design an algorithm (in short, a precise set of steps/actions to solve the problem)
 4. Test the algorithm in your head or by tracing through execution on paper
 5. Write the Python program code in a text editor or IDE (e.g., VS Code)
 6. Execute and test your Python code/program from a command prompt. Possibly repeat any of the previous steps until the program works correctly
-

The first program you will complete is a simple program that prints a phrase ("Hello World") to the console. This program is simple enough that we can largely skip steps 1-4 of the above process. The algorithm for this program is one step: print "Hello World" to the console.

The next step of the process involves writing the necessary code inside a text editor and saving it as a file on the computer. Open VS Code, open a new file (remember to set the language to Python) and **type** the following in the text editor (try to **avoid cut & paste** from a PDF file to a text file, it generally does not work very well).

```
# my first python program
print("Hello World")
```

What is happening here?

- The first line of this code starts with #, which is used to tell Python that you are writing a comment. This means that everything to the right of the # is ignored by the Python interpreter. Comments can be used to explain what a block of code is doing, make note of your thought process, or include any other information that may be useful in understanding the code.
- The second line contains a print command. This command tells the Python interpreter to print whatever text is inside the brackets without the quotes. Note: the quotes indicate that the text is a **String** - more on that later.

- Now that you have written some code in the text editor, you need to save it in a **.py** file before you can execute it and see the result. Save the file as “**p2.py**” wherever you prefer on your computer.
- Run the program using VS Code’s **Run Python file in the Terminal** option that looks something like this . The output should be visible on the console - typically a small panel at the bottom.

That’s all there is to it! You have now created and run a Python program.

- **Optional step:** follow this step only if you prefer to execute Python programs from the command prompt, otherwise ignore this part. Use the Windows Powershell command prompt program. If you are using a Mac computer, the equivalent program is called Terminal (note: you may also need to replace command line references to *python* with *python3* on a Mac) Open the Windows Powershell program, type **ls** (that is, LS in lower case letters) and hit Enter to see which directory (folder) you are currently in and what files are in that directory. The output of the command should start with something like

Directory: C:\Users\yourname

You need to change the directory to be the same as the one where you saved your **p2.py** file. To change directories within Powershell, you use the **cd** command. Typing **cd c:** and hitting enter will change the directory to the C drive. Use the **ls** command to confirm you have changed to the C drive and see what files are there. You should see your **p2.py** file listed – if you do not, make sure you saved it to the directory you are currently in.

Once you are in the same directory as your saved file, type **python p2.py** and hit Enter to run your program in the Python interpreter (Mac users will likely have to type **python3 p2.py**). Doing this runs a separate program, called the Python interpreter, and tells it to execute your Python file p2.py. The Python interpreter will then read and execute each of the instructions you have included inside the specified file (i.e., **p2.py**). If you see the desired output is printed in the Powershell window, your program works as expected!

Problem 3 (User Input)

In the previous problem, you used the print command to print out text. You can perform 'user input' by reading what the user types into the console. The Python command for reading user input from the console is:

```
input("Prompt that you want the user to see on the terminal")
```

Whenever you include this command inside your code, the Python interpreter executing the code will print out the prompt included inside the quotation marks (in this case, 'Prompt that you want the user to see on the terminal') and then wait for the user to type something in and hit the **Enter** button. Whatever text the user types into the console before hitting enter is 'returned' by this command. You can save what the user typed so it can be used later in the program like this:

```
text = input("Type some things: ")
```

This takes the text the user typed and stores it in a 'variable' called **text** (similar to a mathematical expression like, $x = 5$). You can think of this as associating the name **text** with whatever the user typed in. So, any time later in your code where you use the name **text**, you will retrieve the value associated with the name **text**. To see how this works, add the following two lines of code to your **p2.py** file (note: there are no quotation marks around **name** in the second line, variables do not need quotation marks):

```
name = input("Enter your name: ")  
print("Hi,", name, "!")
```

Save the file as **p3.py** and run it again the same way you did for the previous problem. The program should ask you to enter your name and then print your name once you have hit the **Enter** button.

Problem 4 (Submitting in Brightspace)

In this course all tutorial and assignment submissions, unless otherwise stated, will be a single zip file. You must be extremely careful with the details of your submissions.

For example, when you are asked to submit an assignment called **assignment2.zip**, your submission will lose marks if the filename is not exactly as specified. If you submit Assignment2.zip, assignment02.zip, MyAssignment2.zip, a2.zip, Circle.py, or anything that is not exactly assignment2.zip, you will lose marks. Please note that case matters in programming. 'A' and 'a' are not the same!

When you are asked to submit a zip file, you must submit a zip file. This means that if you use 7zip, tar, or any other programs to create a compressed archive of your files you will lose marks. You can create a zip file as follows:

Windows: Select all files you would like in the zip (hold the Ctrl button down to select multiple files or add all the files to a single directory/folder). Right-click on the selected files, select **Send to**, and then **Compressed (zipped) folder**. This will create a zip file of all the files you had selected. You may have to rename this zip file after it is created.

Mac: Put the files you want in your zip file in a directory. Right-click on the directory name and select **Compress "DirectoryName"**, where DirectoryName is the name of the directory you want to create the zip with.

To submit this tutorial, do the following:

- Create a directory called **YourStudentNumber_T1** (replace **YourStudentNumber** with your student number) and copy/move your **p2.py** and **p3.py** files to it. Enter this directory, select both files, and create a zip file containing these files as described above.
- Submit your **YourStudentNumber_T1.zip** file to Brightspace using the submission link in the tutorial section on the Brightspace page.
- After you submit the file, download your submission from Brightspace and confirm that it is a zip file called **YourStudentNumber_T1.zip** that contains two files **p2.py** and **p3.py**. Extract the .py files and execute those again to make sure they work

properly. Occasionally, there can be a problem during the upload process and files can become corrupted. **Downloading, verifying, and executing your submissions ensures that the files were uploaded correctly, which is especially important for assignments as corrupt files cannot be executed or graded.**

Additional Problems

Problem 5 (Basic Problem Solving and Algorithms)

The following few questions will let you practice some simple problem solving and algorithm development skills. Try to analyze each of these problems and design algorithms for their solutions (follow the steps mentioned in Problem 2 on page 6), no need to write code for these problems.

- 1) The sum of 2 consecutive odd numbers is 44. What are the two numbers?
- 2) Assuming that you did not just guess the numbers exactly correct immediately, write out the steps that you used to solve the problem.
- 3) Now assume you are given some number x . Write out general instructions for finding the 2 consecutive odd numbers that sum to x or to determine if no solution exists.
- 4) Assume you are given a list of numbers [8, 3, 5, 1, 2, 9, 7, 6, 4]. Explain in detail the steps you would take to find the largest number. Remember to be specific about things like what numbers you are considering at each step and what actions you are taking (i.e., be much more specific than 'look through the list and find the largest number').
- 5) Explain in detail the steps you would use to find the largest number in a general list of n numbers. What about finding the second largest number?
- 6) Explain in detail the process for drawing a house or some other relatively simple object. Assume we know how to draw the basic shapes like rectangles, circles, squares, ellipse, lines etc.

Problem 6 (Basic Calculations)

- a) You can also perform mathematical calculations using Python. For example, we could create a conversion program that takes a measurement in pounds and prints the equivalent in kilograms:

```
pounds = 22
kilograms = pounds / 2.2
print(kilograms)
```

- b) Try to write a program that calculates the y coordinate of a line given an x value. The general equation for a line is $y = m \cdot x + b$, where m is the slope and b is the y -intercept of the line. Add code to your.py file that sets values for m , b and x and

prints out the associated y value. Try changing the values and checking the correct y value is printed each time.

For a list of mathematical operations that you can use, check out:

http://en.wikibooks.org/wiki/Python_Programming/Basic_Math#Mathematical_Operators

Try adding some additional calculations that use the different mathematical operators to your program.